

Informe de Trabajo Práctico

eSearch21

Autómatas, Teoría de Lenguajes y Compiladores (72.39)

Grupo 59: Tomas Gonzalez Colasantti (Legajo 63281)

Tabla de contenidos

1. Introducción.....	2
2. Modelo computacional.....	2
2.1. Dominio.....	2
2.2. Lenguaje.....	3
3. Implementación.....	4
3.1. Frontend.....	4
3.2. Backend.....	4
3.3. Dificultades encontradas.....	5
4. Futuras extensiones.....	6
5. Conclusiones.....	7

1. Introducción

Este trabajo consistió en elaborar un lenguaje de consulta de documentos basado principalmente en tags asignados a los mismos por los usuarios, como así también por metadatos que representan propiedades intrínsecas de los documentos. Estas consultas luego son traducidas a SQL para ser contestadas por una base de datos.

2. Modelo computacional

2.1. Dominio

El lenguaje desarrollado a lo largo de este trabajo es un DSL orientado a la búsqueda de archivos registrados en una base de datos.

Este lenguaje está pensado para ser utilizado en distintos ámbitos, tanto por programadores como por usuarios finales, tal vez inexpertos. Esto se logra haciendo que sea fácil de abordar, dado que una consulta básica consiste en tags separados por un espacio. Está pensado para tener una forma fácil y rápida de buscar documentos sin depender de estructuras de carpetas, que pueden ser inexactas y confusas, si no de características de cada archivo.

Es un lenguaje implementable en distintos tipos de organizaciones. En el primer informe se dio como ejemplo una empresa que necesita acceder a contratos, facturas, actas y demás documentos burocráticos. Puede ser usado como frontend de búsqueda en alguna página web, como una biblioteca, un foro o un *image board*. También puede implementarse

internamente para uso de un programador, para realizar búsquedas de documentación asociada a distintos proyectos, especificaciones de clientes y demás, o para modificar sus parámetros y usarlo para automatizar la generación manual de consultas SQL a la hora de configurar y testear una base de datos.

2.2. Lenguaje

En su forma más básica, el lenguaje consiste en tags, los cuales son asignados por los usuarios a la hora de cargar o modificar un archivo, a través de otro sistema externo, por ejemplo una GUI. Dichos tags pueden ser cualquier cadena formada por alfanuméricos, guiones medio y bajo, y paréntesis (con la única condición de que no pueden finalizar con un paréntesis de cierre). Una cadena de tags separados por espacio realiza una búsqueda de tipo AND, retornando los archivos que tengan todos esos tags. Para un usuario inexperto, esto es más que suficiente para poder comenzar a usar el lenguaje.

Para usuarios más avanzados, el lenguaje cuenta con capacidad de realizar operaciones de lógica booleana, utilizando '!' para OR y '~' para NOT. Se pueden encerrar predicados entre paréntesis para anidarlos, y se pueden configurar predicados con nombre al inicio de la consulta (utilizando la sintaxis "{ *predicado* }nombre, para luego invocarlos con "*recall:nombre*".

Este lenguaje también tiene metatags, que refieren a propiedades intrínsecas de los archivos, como su tamaño, tipo, nombre, fecha de creación, etc., los cuales pueden ser consultados colocando su nombre, seguido de ':' y el o los parámetros de consulta. Dichos metatags son:

createdon, lasteditedon, editedon, likes, size, views, name, createdby, editedby, lasteditedby, likedby, type, pool,

Soporta ordenar los resultados de consultas según algún criterio específico, mediante el metatag especial “order:”, pudiendo elegir entre: *creationdate, lastedited, likes, size, views*, o *random*.

Por último, el lenguaje soporta wildcards, denotados con ‘*’, para buscar por prefijo o sufijo; rangos, denotados con “*inicio ‘..’ fin*”, y cuantificadores (*>, <, >=, <=*).

Todas estas funcionalidades adicionales hacen que sea un lenguaje de búsqueda potente y versátil para usuarios que lo necesiten, mientras que es lo suficientemente simple como para que un usuario inexperto pueda comenzar a utilizarlo tras unos minutos de explicación.

3. Implementación

3.1. Frontend

El frontend de este lenguaje, basado en el repositorio provisto por la cátedra, utiliza un parser léxico para separar la consulta ingresada en tokens, para luego ser procesada por una gramática del tipo libre de contexto en un analizador sintáctico de tipo LALR(1).

Esta gramática permite armar un árbol sintáctico que indique el orden en el que se deben realizar las distintas operaciones solicitadas para obtener la consulta deseada. Sigue el esquema de expresión -> término -> factor ->

constante. De esta forma, puede aplicar precedencia según lo establecido en la lógica booleana.

Dicho frontend ya realiza las validaciones de tipos de datos necesarias, dado que los enteros, fechas, tamaños de archivo y *cadenas en general* tienen formas distintas, que pueden ser reconocidas por el analizador léxico, por lo que luego el analizador sintáctico puede comparar la entrada obtenida con los tipos requeridos por la gramática, y rechazar la consulta en caso de violarse esta condición.

3.2. Backend

El backend implementado para este compilador-traductor consiste en un analizador semántico que, por un lado, arme una tabla de símbolos para poder reemplazar los predicados con nombre utilizados en la consulta, y por otro lado guarde en esta tabla los distintos tags y metatags consultados junto con su *scope* actual, para poder ejercer el requisito funcional de no permitir tags duplicados.

Este requisito fue ampliado respecto de la primera entrega del trabajo, para ahora no permitir tags con *scopes* solapados. Esto se realizó para, por un lado, mejorar la performance de una consulta, y por otro, para poder utilizar más exhaustivamente la tabla de símbolos antes mencionada.

Una vez validado semánticamente, se pasa a la etapa de generación de código, donde se recorre el árbol sintáctico resultante, y se traduce a la consulta de SQL apropiada para el esquema de base de datos planteado. Este programa está pensado para ser modificado por los administradores de

sistemas que deseen implementarlo, para adecuarlo a los esquemas de DBMS que ya tengan implementados, ya que de lo contrario tendrían que armar una base de datos nueva y realizar las migraciones correspondientes. En caso de que no tengan una base de datos ya configurada, pueden optar por utilizar el esquema ya provisto dentro de este repositorio.

3.3. Dificultades encontradas

Las principales dificultades encontradas a lo largo del desarrollo de este compilador fueron el armado de la gramática, el parsing apropiado de los lexemas y la construcción de la consulta final, es decir, que cada etapa del desarrollo tuvo sus problemáticas que debieron ir solucionándose para llegar a un programa funcional.

Respecto al parsing léxico, los tokens definidos inicialmente tenían mucho *backtracking* a fin de garantizar dos cosas: que únicamente se emitiera un token de metatag o de argumento de metatag si efectivamente se había formado una consulta de metatag válida, y para saber concretamente si un espacio representaba un carácter ignorable, o si representaba un token AND (por ejemplo, en “hola chau”, el espacio representa un token del tipo AND, pero en “hola | chau”, los espacios son ignorables, ya que el token correcto es OR).

Ambos problemas fueron resueltos, haciendo que la gramática maneje los errores en caso de recibir tipos inesperados, lo cual redujo el uso de backtracking a uno o dos casos específicos, y a través de la reescritura de la gramática para no depender del token AND, pudiendo así ignorar todos los espacios recibidos.

Respecto a la gramática utilizada para el análisis sintáctico, el principal problema no fue generar una versión inicial de la misma, si no de adaptarla para el funcionamiento con las nuevas reglas de parsing léxico. Esto se debe a que la gramática originalmente tenía reglas del estilo “expresión -> expresión AND expresión” y “expresión -> expresión OR expresión”, por lo que debían emitirse esos tokens. Los conflictos S/R y R/R resultantes eran resueltos internamente por el parser, utilizando reglas de precedencia y asociatividad ya configuradas.

En un principio, se intentó preservar esa estructura pero removiendo el token AND de la producción correspondiente, lo cual generó numerosos conflictos S/R. Se intentó configurar la misma precedencia establecida anteriormente de forma manual, mediante el uso de la directiva ‘%proc’, pero esto permitía únicamente aplicar precedencia y no asociatividad, por lo cual no resolvió los conflictos. Esto resultó en la reescritura de la gramática (y del resultante árbol sintáctico, generador de código, etc.) para tener nuevas producciones que, por su estructura, garanticen la precedencia y asociatividad necesarias para el correcto funcionamiento del analizador. Terminaron quedando producciones del estilo “expresión -> término OR expresión” para el OR, “término -> factor término” para el AND, y “factor -> NOT constante” para el NOT.

Por último, respecto a la generación de código, el problema consistió en encontrar una forma de realizar una consulta SQL que resulte válida para cualquier input. Esto resultó ser problemático, dado que se puede recibir una mezcla de tags, metatags, y distintas operaciones y niveles de anidamiento. La respuesta encontrada consistió en hacer uso y abuso del operador

EXISTS de SQL para poder encadenar las consultas deseadas, y de esa forma ir anidando estas cláusulas para crear la consulta pedida. Sin embargo, esto resulta poco eficiente, dado que el DBMS tiene que recorrer múltiples veces las distintas tablas para consultas grandes.

Por el momento, esto reduce el dominio de aplicación de este lenguaje a organizaciones con una cantidad moderada de documentos a ingresar al sistema de búsqueda, por lo que a futuro se considera encontrar una mejor forma de realizar estas consultas, o incluir optimizaciones para reducir el costo computacional de la consulta final generada.

4. Futuras extensiones

El lenguaje a futuro puede ser adaptado a distintas salidas, como por ejemplo, para realizar consultas a una base de datos tipo NoSQL, como Mongo o Firebase. Era la idea durante el desarrollo de este trabajo llegar a incluir código de ejemplo para interactuar con un sistema de este estilo, pero debido a limitaciones temporales no se llegó a implementar.

Otra extensión al lenguaje, apuntando a la facilidad de integrarlo en entornos ya existentes, es poder cargar un archivo de configuración que defina la estructura de la base de datos a apuntar, para generar una consulta adecuada para dicho esquema. Esto podría hacerse a través de otro DSL, aunque mucho más sencillo que éste, o a través de un lenguaje ya existente, como JSON o XML. De esta forma, se lograría que en tiempo de ejecución el compilador determine cómo generar la salida, en vez de requerir una recopilación como lo requiere actualmente; además, haría mucho más fácil para un programador adaptarlo a sus necesidades,

ya que resulta muy engorroso ingresar al código fuente y comenzar a modificar las distintas funciones de traducción del árbol sintáctico, sobre todo si no se tiene conocimiento del funcionamiento interno del compilador.

Una última extensión posible sería implementar optimizaciones a la consulta ingresada por el usuario, como la eliminación de tags repetidos o su *elevación de jerarquía* (por ejemplo, que “hola pedro | hola juan” se transforme en “hola (pedro | juan)”), o la condensación de distintos tags del mismo tipo en un único tag equivalente (por ejemplo “createdon:>2000-01-01 createdon:<=2020-01-01” a “createdon:2020-01-02..2020-01-01”). Se dejó programada una tabla de símbolos que registra todos los tags junto con su *scope* correspondiente, y una pila de scopes actuales, lo que debería permitir en un futuro realizar estas optimizaciones.

5. Conclusiones

Para finalizar, se realizó un lenguaje de uso específico y su posterior implementación de analizador y traductor, aplicando todos los conocimientos adquiridos a lo largo de la materia. Dicho lenguaje resultó completamente funcional, pudiéndose correr las consultas SQL generadas contra un motor de PostgreSQL configurado con el esquema de ejemplo, y obteniendo los resultados esperados.

Dicho lenguaje puede resultar, tal vez, algo simple, pero es esa simplicidad lo que puede lograr que sea fácil de utilizar por usuarios con poco conocimiento sobre el mismo, mientras que las funcionalidades más avanzadas que contempla pueden ser aprovechadas por administradores y usuarios con mayor conocimiento informático.

Por otro lado, está pensado para solucionar problemáticas reales, ya que cuando se maneja un gran volumen de archivos a almacenar, puede resultar engorroso el uso de carpetas tradicionales, además de poder olvidar en qué carpeta se encuentra un archivo, o la posibilidad de archivarlo incorrectamente. Con este lenguaje, se puede buscar por distintos tags, para que resulte más fácil encontrar los archivos requeridos.

Por ejemplo, si se guarda una factura y accidentalmente no se le asigna el tag correspondiente al cliente, se puede buscar por factura y ordenar cronológicamente para encontrar la correcta, o si se quiere consultar toda la información de un cliente se puede buscar por su tag, y así tener tanto facturas como otro tipo de documentos (siempre y cuando los mismos sean correctamente catalogados a la hora de ingresarlos al sistema). Con un sistema de archivos y carpetas, se podría realizar únicamente una de las dos operaciones con facilidad, dependiendo de si se agruparon todas las facturas en una carpeta o toda la información de clientes en carpetas. Se podrían realizar distintas copias, pero sería un problema a la hora de mantener la integridad entre los datos si se desea modificar algún archivo.

En conclusión, se diseñó lenguaje funcional con aplicaciones reales y se realizó su implementación programática exitosamente.